B

Fraud Engine × Google CEL

Rules-as-data, decision policy, and cohort routing with Common Expression Language — a shadow-first prototype.

ARGUS FINANCIAL PARTNERS · CONFIDENTIAL — FOUNDER · **JUNE 24, 2026**

Strategic guidance only — not legal, financial, tax, or investment advice. Securities, money-transmission, and corporate-formation decisions must be reviewed by licensed counsel and a CPA before you act. Sections flagged "⚖ see counsel" are where this matters most.

Contents

Fraud Engine × Google CEL — rules-as-data, decision policy, cohort routing

A review of the Argus Fraud Engine architecture and how **Google CEL** (Common Expression Language, cel.dev) makes it better — plus a working prototype that ships CEL rules as a **shadow** model with zero risk to the live decision.

> **How to read this.** Companion to `FraudEngine.md` (the platform design) and `FraudEngine–GapAnalysis.md` (the "this maps to the real production component" interface map). Same framing: every seam has a simulated/in-process default and a documented production swap.

---

1 1. Why CEL fits this engine specifically

The engine is already shaped for CEL — three properties make it an unusually clean fit:

- **A clean activation object.** `enrich()` (`src/features/enrichment.ts`) already turns each event into a flat `EnrichedEvent` — `velocity`, `newPayee`, `trailingMaxMinor`, `avgOutMinor`, `geoChanged`, `deviceChanged`, `amountMinor`, `features.transferOutCount`. That is *exactly* the variable bindings a CEL expression evaluates against; no new capture layer is needed.
- **Rules are the one hardcoded layer.** `RulesModel.score()` (`src/models/rulesModel.ts`) is a wall of `if (cond) add(code, weight)` with magic thresholds. Tuning a threshold or adding a typology is a **code deploy** today. This is the textbook thing CEL turns into **data**.
- **The promotion machinery already exists.** A model is `Model { version, kind, score(ev) }` registered in `ModelServer`, with a DB-backed status (`prod | shadow | canary | retired`) the `Router` reads live (`src/models/registry.ts`, `src/router/router.ts`). So **a CEL ruleset is just another model version** — it inherits shadow-testing, canary %, promote-with-no-redeploy, the `fe_shadow_divergence_total` metric, and the append-only `decisions` audit **for free**. That is the synergy that makes CEL worth it here rather than a generic "add a rules engine."

What CEL buys you over hand-rolled `if`s:

Property	Hardcoded `RulesModel`	CEL ruleset
Authoring	Engineer, in TS	**Analyst, as data** (a `{code, expr, weight}` row)
Change cadence	Code review + deploy	**Hot reload / registry promote — no redeploy**
Safety	Arbitrary TS (can loop, throw, reach IO)	**Sandboxed, non-Turing-complete, bounded**
Portability	Node-only	**A spec** — same expr runs on cel-go/Java/Python/C++
Explainability	`{code, weight}` per rule	**Identical** — every fired rule still emits a Reason/Contribution
Rollout	All-or-nothing	**Shadow → canary → prod** on the existing registry

2 2. The three CEL applications (all prototyped)

2.1 Rules-as-data (the headline)

`RulesModel` → a `rules` table of `{code, CEL expr, weight}` evaluated by a `CelRulesModel` that implements the same `Model` interface and emits the **same** `ModelOutput` / `Reason` / `Contribution`. An analyst adds the "structuring just under $10k" rule by inserting a row (`expr: "amountMinor >= 800000 && amountMinor < 1000000"`), not by editing code. The default ruleset is a **faithful port of all of rules-v1** (proven by a parity test), so adoption starts at exact equivalence.

2.2 Decision policy

`actionFor()`'s fixed `block/challenge/flag/freeze` ladder → an `action_policy` table of `{action, expr, priority}`. The highest-priority matching CEL row decides — e.g. `block if score >= 800 || ('sanctions_hit' in reasonCodes && amountMinor > 500000)`. Opt-in (`ACTION_POLICY=cel`); the threshold ladder stays the safe default, and the sync path still can't freeze.

2.3 Routing / canary cohort predicates

Canary targeting by a CEL **cohort predicate** (`models.cohort_expr`, e.g. `channel == 'card' && geo == 'NG'`) instead of only a hash percentage — so a new ruleset can be canaried to *card payments in a specific corridor* before a global rollout. Null predicate = today's percentage-only behavior.

---

3 3. The production map (where CEL sits)

`FraudEngine-GapAnalysis.md` maps each layer to its real-world component (Kafka, Flink, Triton, MLflow). CEL fills the **rule/policy authoring layer** that map didn't name:

Engine layer	This prototype	Production swap
Rule evaluation	In-process CEL **subset** (`RULE_EVALUATOR=subset`)	**cel-go** behind a gRPC sidecar / WASM (`RULE_EVALUATOR=celgo`) — spec-complete + fastest
Rule authoring/storage	`rules` table seeded from code	A rule-editor UI + approval workflow over the same table
Rollout	model registry (shadow/canary/prod)	unchanged — the registry IS the rollout control

This mirrors how mature platforms externalize rules (Sift/Unit21 rule editors; OPA/CEL for policy in k8s/Envoy/IAM). CEL is the same idea applied to the fraud decision.

---

4 4. Risks & how the design handles them

- **Node CEL maturity.** The spec-complete impls are Go/C++/Java/Python; Node has only community libs. So the evaluator is a **swappable** `RuleEvaluator` **seam** with an in-process **CEL-compatible subset** now and **cel-go** as the production swap. Expressions are written in CEL syntax so they run *unchanged* on cel-go later. We deliberately did **not** add a heavyweight dependency (the engine keeps its zero- expression-lib footprint).
- **Honesty about "CEL".** The in-process evaluator supports a strict subset (field access, comparisons, `&& || !`, arithmetic, `in`, ternary, `size` / `has`) and **omits loops, comprehensions, and user functions** — so it is non-Turing-complete and bounded by AST size. It is not a claim of full CEL in Node; it is a forward-compatible subset.

- **Blocking-path latency.** Expressions **compile once** (program cache) at ruleset load; scoring only evaluates, which is bounded and cannot throw for control flow (a bad expression fails at *compile*, is skipped, and is surfaced via `compileErrors` — a bad analyst rule never takes down scoring).
- **`bigint → int64` boundary.** Money is `bigint` minor units; CEL ints are int64. Minor-unit amounts fit in int64 and within JS's exact-integer range, so the conversion is lossless; documented in `src/rules/activation.ts`.
- **Money-path determinism.** `rules-v1` stays **prod**; CEL ships **shadow-first** and never changes the live decision until someone promotes it. The engine is standalone (Argus calls it over HTTP), so there is zero Argus money-path risk.

---

5 5. What the prototype proves (and how it's verified)

Built in `fraud-engine/` (`src/rules/celEvaluator.ts` , `src/rules/activation.ts` , `src/models/celRulesModel.ts` , `src/router/decisionPolicy.ts` ; seeded via `src/rules/rulesetStore.ts` ; wired in `src/context.ts`), with `test/cel-rules.test.ts` (15 tests). The suite asserts:

1. **Evaluator correctness + safety** — operators/precedence/membership; a malformed expression throws at **compile**, not on the hot path; comprehension/loop syntax is rejected (bounded).
2. **Parity** — `rules-cel-v1` produces the **same score + reasons** as the hardcoded `rules-v1` across benign / large-absolute / structuring / pass-through / geo+device scenarios. The CEL port is faithful.
3. **Hot reload** — `UPDATE`-ing a rule's `weight` / `expr` row changes scoring with **no code edit** (the "analyst tunes a rule without a deploy" proof).
4. **Registry adoption** — `rules-cel-v1` registers as **shadow**, is scored alongside prod on a real `engine.process()` event, and `registry.promote(…, "prod")` flips it live **with no restart**.
5. **Decision policy** — a CEL `action_policy` row maps `score → action` ; no match falls back to thresholds.
6. **Cohort routing** — a `cohort_expr` gates canary activation (a card-channel event gets the CEL canary; an API-channel event does not).

Run it: `cd fraud-engine && npm run typecheck && npm test` (44 pass / 9 files; the existing suite is unchanged).

6 6. The cutover path

1. **Now (shipped):** CEL ruleset live as **shadow**; divergence vs. rules-v1 visible on `fe_shadow_divergence_total`. Author new rules as data; watch them in shadow.
2. **Canary:** `registry.promote("rules-cel-v1", "canary", pct)` (+ optional `cohort_expr`) — a slice of traffic decides on CEL; the rest stays on rules-v1.
3. **Prod:** promote to `prod`; retire `rules-v1` when confidence holds. Add a rule-editor UI over the `rules` / `action_policy` tables.
4. **cel-go swap:** when rule volume/latency or cross-language parity demands it, register a `celgo` `RuleEvaluator` (gRPC sidecar / WASM) behind the same interface — the CEL expressions are unchanged.

Companion documents: `FraudEngine.md` *(platform design),* `FraudEngine-GapAnalysis.md` *(production interface map). Prototype:* `fraud-engine/src/rules/*`, `src/models/celRulesModel.ts`, `src/router/decisionPolicy.ts`, `test/cel-rules.test.ts`.